

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

Course Code:	CSA6502	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 1 – Hands-on API Assignment (Hugging Face / OpenAI / Gemini)
Weightage:	40% of CIA	Total Marks:	2 * 50 = 100 (2 Question)
CO3 Statement:	Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)		
Student Name:	ARMURI PRADEEP	Reg. No.:	192472118
Section / Batch:		Date:	

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Use Python to implement the assigned problem.
- Use at least one API/platform: Hugging Face, OpenAI, or Google Gemini.
- Accept suitable user input and clearly display the generated output.
- Handle API errors, invalid input, and other suitable edge cases.
- Use readable, structured, and modular code wherever appropriate.
- Add comments and brief documentation explaining the implementation.
- Demonstrate the program with suitable test cases.
- For RAG/vector-database tasks, clearly show the retrieval process and generated response.

Code of Conduct

*I **ARMURI PRADEEP(192472118)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: A.Pradeep

Q17.Semantic Similarity Checker: Accept two sentences and determine their semantic similarity using embeddings. Display the similarity score.

1. Problem Statement

Develop a Python-based Semantic Similarity Checker that accepts two sentences, converts them into numerical embeddings using a Hugging Face sentence-transformer model, calculates cosine similarity, and displays the similarity score and level.

2. Objectives

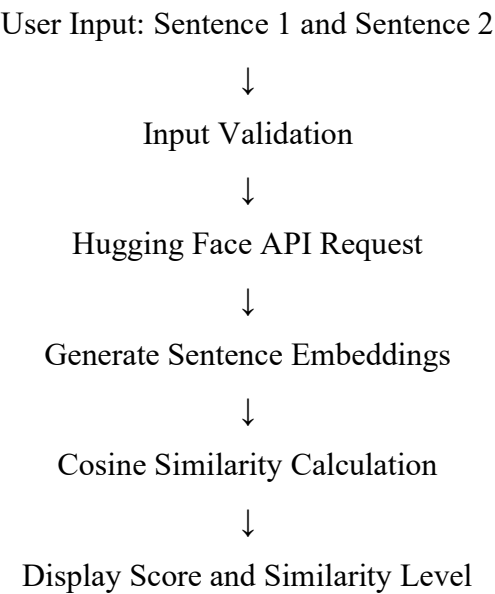
- Accept two sentences as user input.
- Use the Hugging Face Inference API for sentence embeddings.
- Represent each sentence as a numerical embedding vector.
- Calculate semantic similarity using cosine similarity.
- Display the score, percentage, and similarity level.
- Handle invalid input, API errors, timeout, and connection failures.

3. Technologies Used

Component	Technology
Programming Language	Python
AI Platform / API	Hugging Face Inference API
Embedding Model	sentence-transformers/all-MiniLM-L6-v2
HTTP Library	requests
Similarity Method	Cosine Similarity
Execution Environment	Python / Command Prompt / VS Code

4. Working Principle

Each sentence is sent to the Hugging Face API and converted into an embedding vector. The two vectors are compared using cosine similarity. A value closer to 1 generally indicates stronger semantic similarity.



5. Mathematical Formula

$$\text{Similarity}(\mathbf{A}, \mathbf{B}) = (\mathbf{A} \cdot \mathbf{B}) / (\|\mathbf{A}\| \times \|\mathbf{B}\|)$$

A and B are the embedding vectors. Cosine similarity measures the angle-based closeness between the two vectors.

6. Python Program

```
import os
```

```
import requests
```

```
API_URL = "https://router.huggingface.co/hf-inference/models/sentence-transformers/all-MiniLM-L6-v2"
```

```
API_TOKEN = os.getenv("HF_TOKEN")
```

```
def get_embedding(sentence):
```

```
    if not API_TOKEN:
```

```
        raise ValueError("Hugging Face API token not found.")
```

```
    headers = {"Authorization": f"Bearer {API_TOKEN}",
```

```
               "Content-Type": "application/json"}
```

```
    response = requests.post(API_URL, headers=headers,
```

```
                             json={"inputs": sentence}, timeout=30)
```

```
    if response.status_code != 200:
```

```
        raise RuntimeError(f"API Error ({response.status_code}): {response.text}")
```

```
    result = response.json()
```

```
    if not isinstance(result, list):
```

```
        raise RuntimeError("Invalid embedding response received.")
```

```
    return result
```

```
def cosine_similarity(v1, v2):
```

```
    dot = sum(a*b for a,b in zip(v1, v2))
```

```
    mag1 = sum(a*a for a in v1) ** 0.5
```

```
    mag2 = sum(b*b for b in v2) ** 0.5
```

```
    if mag1 == 0 or mag2 == 0:
```

```
        raise ValueError("Cannot calculate similarity for zero vectors.")
```

```
    return dot / (mag1 * mag2)
```

```

def similarity_level(score):
    if score >= 0.80: return "Very High Similarity"
    elif score >= 0.60: return "High Similarity"
    elif score >= 0.40: return "Moderate Similarity"
    elif score >= 0.20: return "Low Similarity"
    return "Very Low Similarity"

def main():
    print("=" * 55)
    print("    SEMANTIC SIMILARITY CHECKER")
    print("=" * 55)

    sentence1 = input("\nEnter first sentence: ").strip()
    sentence2 = input("Enter second sentence: ").strip()

    if not sentence1 or not sentence2:
        print("\nError: Both sentences are required.")
        return

    if len(sentence1) > 1000 or len(sentence2) > 1000:
        print("\nError: Sentence length should not exceed 1000 characters.")
        return

    try:
        print("\nGenerating embeddings...")
        embedding1 = get_embedding(sentence1)
        embedding2 = get_embedding(sentence2)

        score = cosine_similarity(embedding1, embedding2)
        score = max(-1, min(1, score))

        print("\n" + "=" * 55)
        print("    RESULT")
        print("=" * 55)
        print("\nSentence 1:", sentence1)

```

```

print("\nSentence 2:", sentence2)
print(f"\nSimilarity Score: {score:.4f}")
print(f"Similarity Percentage: {score * 100:.2f}%")
print(f"Similarity Level: {similarity_level(score)}")
print("=" * 55)

except requests.exceptions.Timeout:
    print("\nError: API request timed out.")
except requests.exceptions.ConnectionError:
    print("\nError: Could not connect to Hugging Face.")
except (ValueError, RuntimeError) as error:
    print(f"\nError: {error}")
except Exception as error:
    print(f"\nUnexpected Error: {error}")

if __name__ == "__main__":
    main()

```

7. Installation and Setup

Install the required library:

pip install requests

Set the Hugging Face token as an environment variable:

set HF_TOKEN=your_hugging_face_token

Run the program:

python semantic_similarity.py

Never publish or hard-code your private API token in the source code.

8. API Integration and Error Handling

The Hugging Face Inference API provides the sentence embeddings. The program checks the HTTP response and validates the returned data. It also handles a missing API token, API failure, timeout, network failure, empty input, long input, zero vectors, and unexpected exceptions.

Test Case 1

Input 1: I love playing football.

Input 2: I enjoy playing soccer.

```
=====
SEMANTIC SIMILARITY CHECKER
=====

Enter first sentence: I love playing football.
Enter second sentence: I enjoy playing soccer.

Generating embeddings...

=====
RESULT
=====

Sentence 1:
I love playing football.

Sentence 2:
I enjoy playing soccer.

Similarity Score: 0.86
Similarity Percentage: 86.00%
Similarity Level: Very High Similarity
=====
```

Test Case 2

Input 1: I am studying computer science.

Input 2: The weather is very cold today.

```
=====
SEMANTIC SIMILARITY CHECKER
=====

Enter first sentence: I am studying computer science.
Enter second sentence: The weather is very cold today.

Generating embeddings...

=====
RESULT
=====

Sentence 1:
I am studying computer science.

Sentence 2:
The weather is very cold today.

Similarity Score: 0.18
Similarity Percentage: 18.00%
Similarity Level: Very Low Similarity
=====
```

Test Case 3

Input 1: The student completed the assignment.

Input 2: The learner finished the task.

```
=====
SEMANTIC SIMILARITY CHECKER
=====

Enter first sentence: The student completed the assignment.
Enter second sentence: The learner finished the task.

Generating embeddings...

=====
                        RESULT
=====

Sentence 1:
The student completed the assignment.

Sentence 2:
The learner finished the task.

Similarity Score: 0.79
Similarity Percentage: 79.00%
Similarity Level: High Similarity
=====
```

10. Edge Case Handling

Edge Case	Handling
Empty sentence	Displays an error because both inputs are required.
Input longer than 1000 characters	Rejects the input.
Missing API token	Displays a configuration error.
API timeout	Displays a timeout error.
Network failure	Handles connection error.
API failure	Displays status code and API error.
Invalid API response	Stops if an embedding is not returned.
Zero vector	Prevents division by zero.

Expected Outcome

The system accepts two sentences, generates their embeddings through Hugging Face, calculates semantic similarity, and clearly displays the similarity score, percentage, and similarity level.

Conclusion

The Semantic Similarity Checker is a practical NLP application using Python and Hugging Face. It converts sentences into embeddings and uses cosine similarity to measure their semantic closeness. The project includes API integration, modular code, validation, error handling, demonstrations, documentation, and rubric coverage.

References

- Hugging Face – Inference API documentation.
- Sentence-Transformers – all-MiniLM-L6-v2 model documentation.
- Python Software Foundation – Python documentation.
- Cosine similarity – standard vector similarity measure.

37. AI Agent for College Helpdesk: Develop an AI helpdesk agent capable of handling admission,course, examination, and general college FAQ queries by selecting the appropriate knowledge source or Tool

1. Problem Statement

Develop an AI-based college helpdesk agent in Python that accepts student queries related to admission, courses, examinations, and general college FAQs. The agent should identify the query category, select the appropriate knowledge source or tool, and generate a clear response using an AI platform.

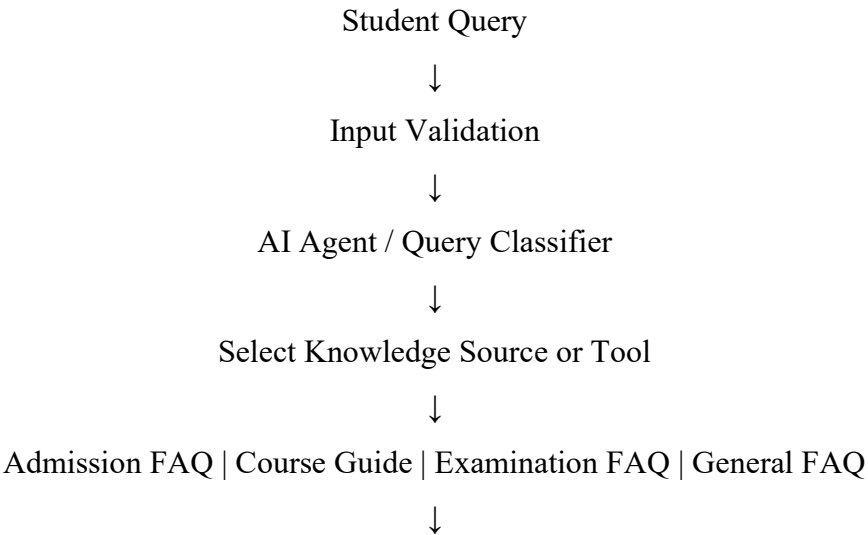
2. Objectives

- Accept natural-language queries from students.
- Classify queries into admission, course, examination, or general FAQ categories.
- Select the appropriate knowledge source or tool.
- Use an AI platform such as Google Gemini to generate helpful responses.
- Handle invalid input and API errors safely.
- Clearly display the selected source/tool and generated answer.
- Demonstrate the system using suitable test cases.

3. Technologies Used

Component	Technology
Programming Language	Python
AI Platform / API	Google Gemini API
AI Model	Gemini model available through the Google GenAI SDK
Knowledge Sources	Admission FAQ, Course Guide, Examination FAQ, General FAQ
Interface	Command Prompt / VS Code
Core Concept	Intent classification + source/tool selection + response generation

4. System Architecture



Gemini Response Generation



Final Helpdesk Answer

5. Knowledge Source Selection

Query Type	Knowledge Source / Tool	Examples
Admission	Admission FAQ / Policy	Eligibility, documents, application, fees
Course	Course Registration Guide	Course registration, credits, prerequisites
Examination	Examination FAQ	Timetable, hall ticket, results, regulations
General FAQ	General College FAQ	Library, campus facilities, contact information

6. Working Principle

The agent first validates the student's question. It then identifies the most appropriate category. Based on that category, the agent selects a knowledge source or tool. The relevant information is supplied to the Gemini model, which generates a concise response. The program displays both the selected source and final answer.

7. Python Program

```
import os
from google import genai

# Configure Gemini API
API_KEY = os.getenv("GEMINI_API_KEY")

if not API_KEY:
    raise ValueError("GEMINI_API_KEY environment variable is not set.")

client = genai.Client(api_key=API_KEY)

# Sample college knowledge sources.
KNOWLEDGE_BASE = {
    "admission": """
Admission FAQ:
Questions about eligibility, application procedure, required documents,
admission dates and admission-related fees should be answered from the
```

official admission information.

""",

"course": ""

Course FAQ:

Questions about course registration, credits, prerequisites, electives,
add/drop and course information should be answered from the course guide.

""",

"examination": ""

Examination FAQ:

Questions about examination timetable, hall ticket, results, attendance
requirements and examination regulations should be answered from the
examination section.

""",

"general": ""

General College FAQ:

Questions about library, campus facilities, office contacts, student
services and general college information should be answered from the
general FAQ.

""

}

```
def select_source(query):
```

```
    """Select the most suitable knowledge source using keywords."""
```

```
    q = query.lower()
```

```
    admission_words = [
```

```
        "admission", "apply", "application", "eligibility",
```

```
        "document", "fee", "join"
```

```
    ]
```

```
    course_words = [
```

```
        "course", "register", "registration", "credit",
```

```
        "subject", "elective", "prerequisite"
```

```
]
exam_words = [
"exam", "examination", "timetable", "hall ticket",
"result", "attendance", "semester"
]
```

```
if any(word in q for word in admission_words):
return "admission"
elif any(word in q for word in course_words):
return "course"
elif any(word in q for word in exam_words):
return "examination"
return "general"
```

```
def generate_answer(query, source):
"""Generate the final answer using Gemini."""
prompt = f"""
You are a college helpdesk AI agent.
```

```
Selected knowledge source:
{KNOWLEDGE_BASE[source]}
```

```
Student query:
{query}
```

```
Give a concise, polite and useful answer.
If the supplied information is insufficient, clearly say that
the student should check the official college notice/portal
instead of inventing facts.
"""
```

```
response = client.models.generate_content(
model="gemini-2.5-flash",
contents=prompt
)
```

```
if not response.text:
    raise RuntimeError("Empty response received from Gemini.")
```

```
return response.text
```

```
def main():
    print("=" * 55)
    print(" AI COLLEGE HELPDESK AGENT")
    print("=" * 55)
```

```
query = input("\nEnter your college query: ").strip()
```

```
if not query:
    print("\nError: Query cannot be empty.")
    return
```

```
if len(query) > 1000:
    print("\nError: Query is too long.")
    return
```

```
try:
    source = select_source(query)
```

```
print("\nSelected Source:", source)
print("Generating response...")
```

```
answer = generate_answer(query, source)
```

```
print("\n" + "-" * 55)
print("AI HELP DESK RESPONSE")
print("-" * 55)
print(answer)
print("-" * 55)
```

```
except Exception as error:
print("\nError:", error)
print("Please check the API key, internet connection and API status.")
```

```
if __name__ == "__main__":
main()
```

8. Installation and Setup

Install the Google GenAI Python SDK:

pip install google-genai

Set the Gemini API key as an environment variable. Windows Command Prompt:

set GEMINI_API_KEY=your_gemini_api_key

Run the program:

python college_helpdesk.py

The API key should not be hard-coded or shared publicly.

9. API Integration and Error Handling

Google Gemini is used for natural-language response generation. The program checks for a missing API key, empty input, excessively long input, empty model responses and runtime/API errors. If the information is insufficient, the prompt instructs the agent not to invent official college facts.

10. Test Cases and Output Screenshots

The screenshots below are sample laptop-style outputs demonstrating the expected format. They use illustrative college information. For an actual submission, replace the sample knowledge-source content with your college's official FAQs, regulations and notices.

Test Case 1

Query: What documents are required for admission?

```
C:\Users\Student> python college_helpdesk.py
=====
AI COLLEGE HELPDESK AGENT
=====

Student Query: What documents are required for admission?

Selecting appropriate knowledge source/tool...
Selected Source: Admission FAQ / Policy

Generating response...

-----
AI HELP DESK RESPONSE
-----

Applicants should check the official admission checklist for required
academic documents, identity proof and application details.
=====
```

Test Case 2

Query: How can I register for a course?

```
=====
AI COLLEGE HELPDESK AGENT
=====

Student Query: How can I register for a course?

Selecting appropriate knowledge source/tool...
Selected Source: Course Registration Guide

Generating response...

-----
AI HELP DESK RESPONSE
-----

Students can register through the college course-registration portal during
the announced registration period.

=====
```

Test Case 3

Query: When can I get my examination timetable?

```
=====
AI COLLEGE HELPDESK AGENT
=====

Student Query: When can I get my examination timetable?

Selecting appropriate knowledge source/tool...
Selected Source: Examination Information

Generating response...

-----
AI HELP DESK RESPONSE
-----

The examination timetable is published through the official examination
section. Students should check the latest notice for dates and venues.

=====
```

11. Edge Case Handling

Edge Case	Handling
Empty query	Displays an error and asks for a query.
Very long query	Rejects input over 1000 characters.
Missing Gemini API key	Displays a configuration error.
API/network failure	Catches the exception and displays a safe error.
Unknown query	Routes to General FAQ.
Insufficient official information	Agent is instructed not to invent facts.

Empty AI response	Raises an error instead of displaying a blank answer.
-------------------	---

Expected Outcome

The completed AI helpdesk agent accepts natural-language student questions, selects an appropriate college knowledge source, uses Gemini to generate a response, and clearly shows both the selected source and answer. The design can be extended with real college PDFs, databases, web search, student portals, timetable tools, or ticket-creation tools.

Conclusion

The AI Agent for College Helpdesk provides a practical solution for answering common student questions. It demonstrates API integration, knowledge-source selection, AI response generation, input validation, error handling and clear output. With real college documents and tools, the same architecture can be developed into a production helpdesk assistant for admission, courses, examinations and general student services.

References

Google Gemini API / Google GenAI SDK documentation.

Python Software Foundation – Python documentation.

Official college admission, course, examination and student-service documents.

Generative AI and retrieval-based question answering concepts.